

02 Matrices



2.4 Matrix Multiplication

Shape Matching, Row-Column Dot Products, and the @ Operator in NumPy

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills for This Section

1

Understand matrix multiplication: two matrices multiply to produce a new matrix, $C = AB = A @ B$

3

Compute matrix products with NumPy: the @ operator, np.matmul(), and np.dot()

5

Recognize that matrix multiplication can degenerate to a scalar: a 1×1 matrix product

2

Master the shape-matching rule: the left matrix's columns must equal the right matrix's rows

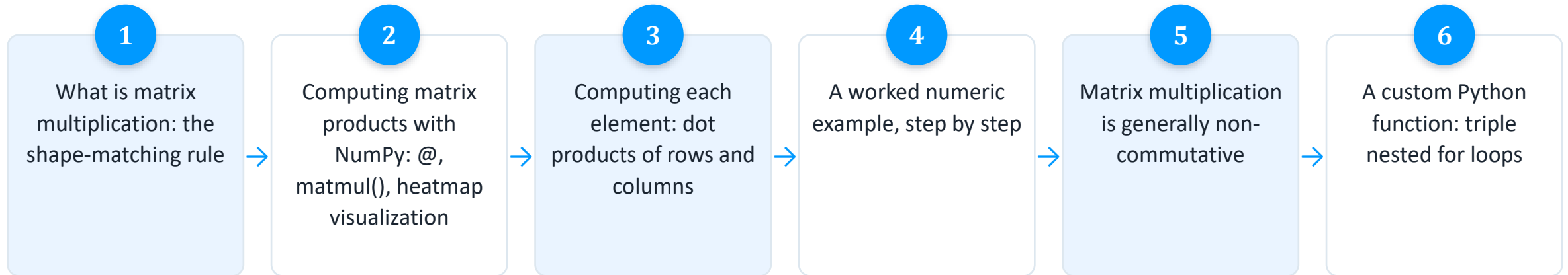
4

Understand how each product element is computed: the dot product of a row and a column vector

6

Understand that matrix multiplication is generally non-commutative, $AB \neq BA$

What This Section Covers



What Is Matrix Multiplication?

Matrix multiplication combines two matrices to produce a new matrix

- Matrix multiplication may be the single most important operation in linear algebra; shape matching deserves special attention
- The result of matrix multiplication can sometimes be a 1×1 matrix, which we treat as a scalar

Figure 1 · Shapes of Matrices A and B

- Matrix **A** has shape $m \times p$ (m rows, p columns); matrix **B** has shape $p \times n$ (p rows, n columns)
- The number of columns in **A** must equal the number of rows in **B**, both p — the prerequisite for multiplication

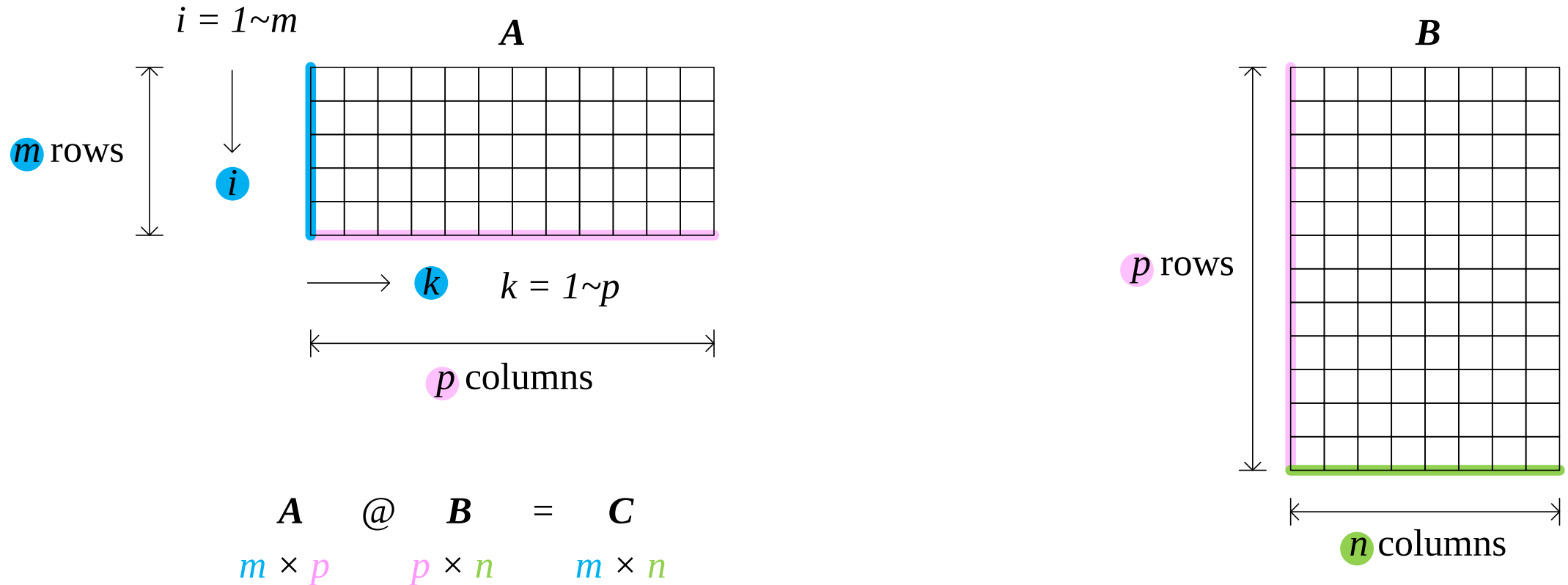
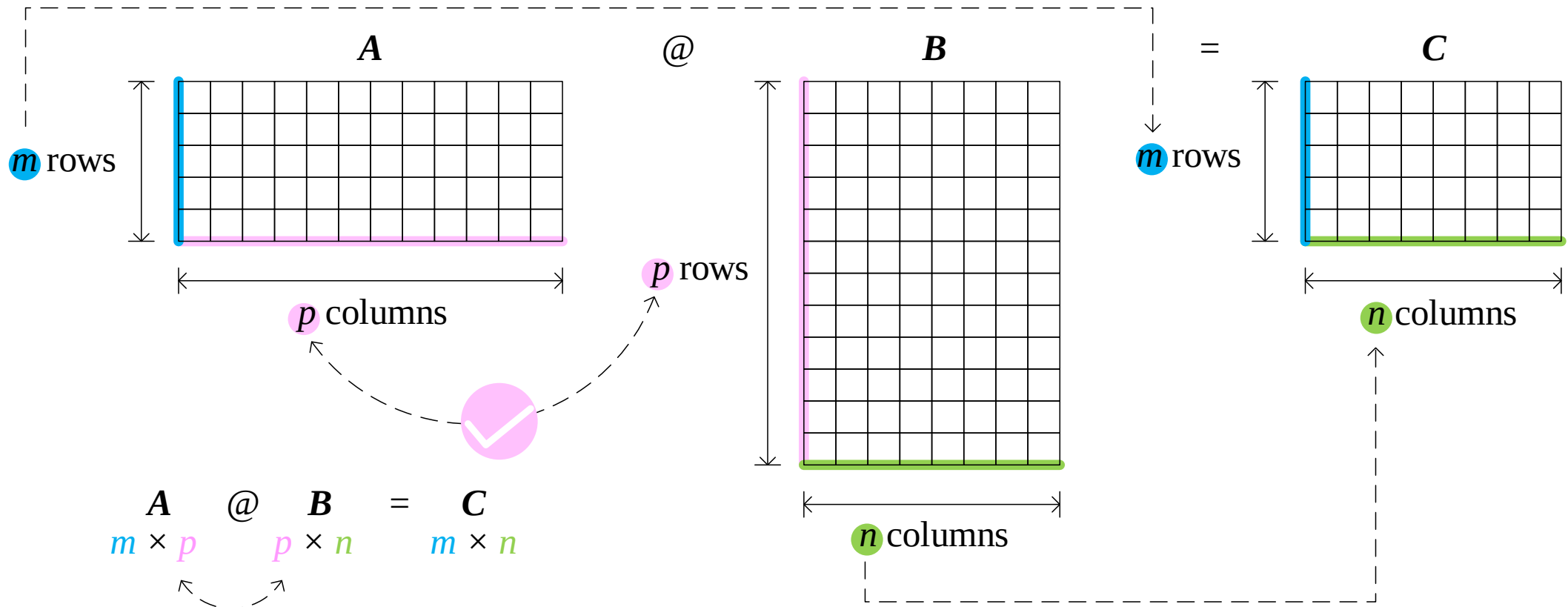


Figure 2 · Matrix Product $A @ B$

- The product $C = AB = A @ B$ is a matrix with shape $m \times n$
- This book also writes AB as $A @ B$, matching the syntax used by NumPy and other Python libraries



Shape Matching: the Middle Dimension Cancels

$(m \times p) @ (p \times n)$ — the p in the middle cancels, leaving shape $m \times n$

- $\mathbf{C}$ inherits $\mathbf{A}$'s m rows and $\mathbf{B}$'s n columns; the shared p in the middle is effectively “canceled out”
- If the left matrix's columns do not equal the right matrix's rows, the multiplication cannot be performed

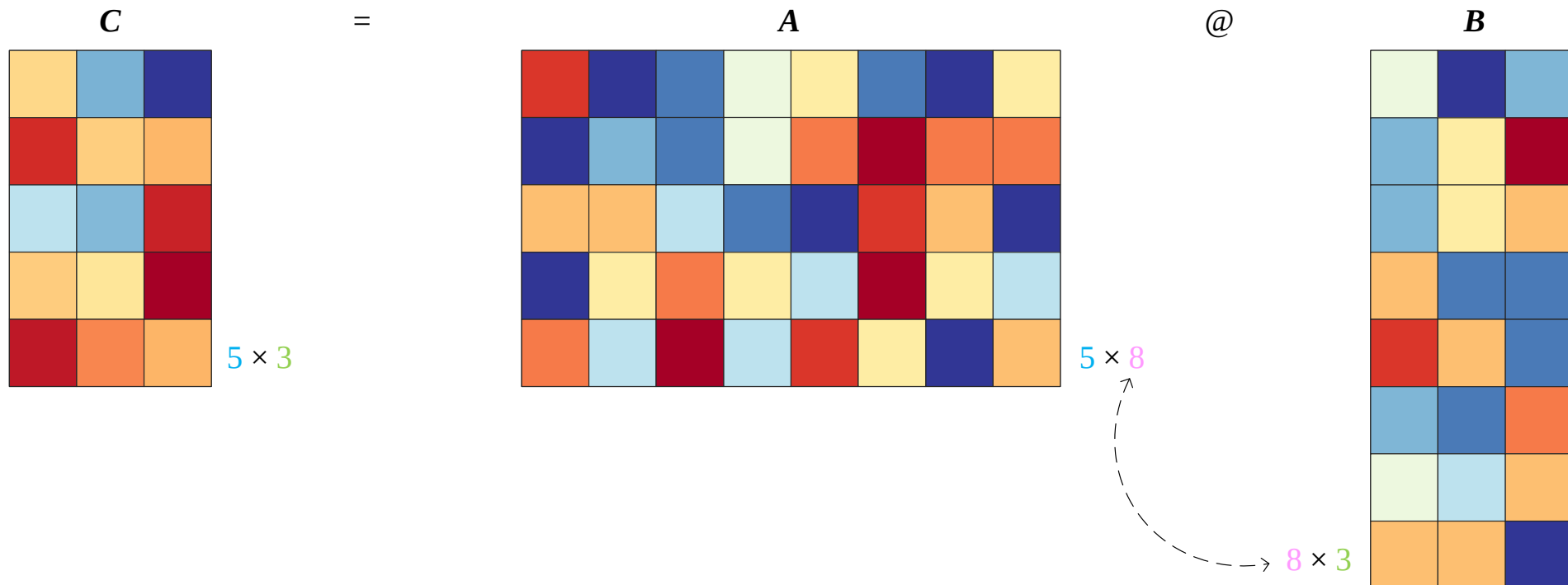
Computing Matrix Products with NumPy

NumPy offers three ways: the @ operator, `np.matmul()`, or `np.dot()`

- Code 1 uses the @ operator and visualizes the product as a heatmap
- $(5 \times 8) @ (8 \times 3)$ — the middle 8 cancels, leaving a result of shape (5×3)

Figure 3 · Visualizing the Product $A @ B$ as a Heatmap

- `seaborn.heatmap()` renders C , A , and B as color matrices, making their shapes easy to compare at a glance
- Color intensity encodes value magnitude — no need to read every number to grasp the overall pattern



Visualizing Matrix

Multiplication as a

Heatmap

Compute $A @ B$ with the $@$ operator, then display C , A , B side by side as heatmaps

- `np.random.seed(88)` fixes the seed;
`np.random.randint()` creates A with shape (5,8) and B with shape (8,3)
- `C = A @ B` performs the multiplication in one line, equivalent to `np.matmul(A, B)`
- `plt.subplots(1,5,...)` creates a 1×5 grid of subplots to hold C , an equal sign, A , an $@$ sign, and B
- `sns.heatmap()`'s `cmap`, `xticklabels`, `yticklabels`, and `cbar_kws` arguments control the heatmap's appearance

```
## 初始化
import numpy as np
import seaborn as sns
from matplotlib import pyplot as plt

## 创建矩阵A和B
a np.random.seed(88)
A = np.random.randint(0, 10, (5, 8))
B = np.random.randint(0, 10, (8, 3))

## 矩阵乘法 A@B
b C = A @ B

## 可视化矩阵乘法
c fig, axs = plt.subplots(1, 5, figsize=(8, 3))

d plt.sca(axs[0])
e ax = sns.heatmap(C, cmap='RdYlBu_r',
                  xticklabels = [], yticklabels = [],
                  cbar_kws={"orientation": "horizontal"})
f ax.set_aspect("equal")
g plt.title('C')

plt.sca(axs[1])
plt.title('=') # 绘制等号
h plt.axis('off')

plt.sca(axs[2])
ax = sns.heatmap(A, cmap='RdYlBu_r',
                  xticklabels = [], yticklabels = [],
                  cbar_kws={"orientation": "horizontal"})
ax.set_aspect("equal")
plt.title('A')

plt.sca(axs[3])
plt.title('@') # 绘制乘号
plt.axis('off')

plt.sca(axs[4])
ax = sns.heatmap(B, cmap='RdYlBu_r',
                  xticklabels = [], yticklabels = [],
                  cbar_kws={"orientation": "horizontal"})
ax.set_aspect("equal")
plt.title('B')
```



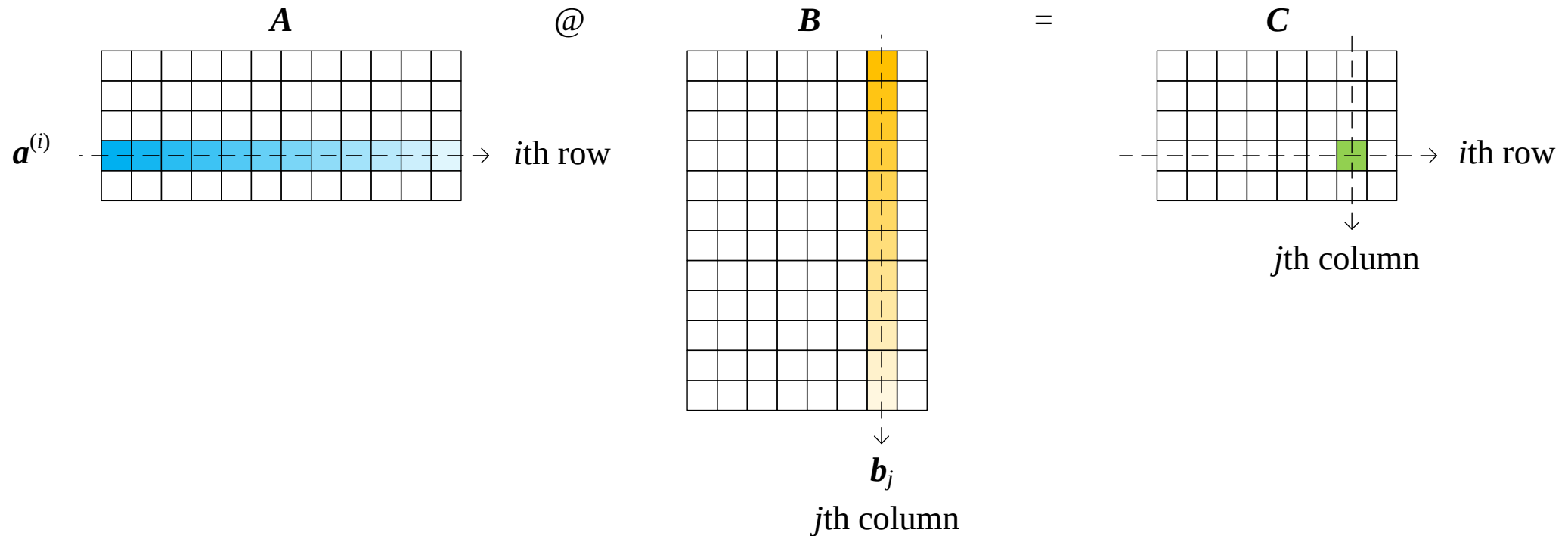
Computing Each Element

The element in row i , column j of C is the sum of A 's row i multiplied elementwise by B 's column j

- This summation is exactly the dot product of $\mathbf{A}$'s i -th row vector and $\mathbf{B}$'s j -th column vector
- The dot product of a row vector and a column vector is the core operation behind matrix multiplication

Figure 4 · Computing Each Element of $C = AB$

- Take A 's i -th row (blue) and B 's j -th column (orange), multiply elementwise and sum, to get C 's (i, j) entry (green)
- Sweeping through every combination of i and j completes the full matrix multiplication



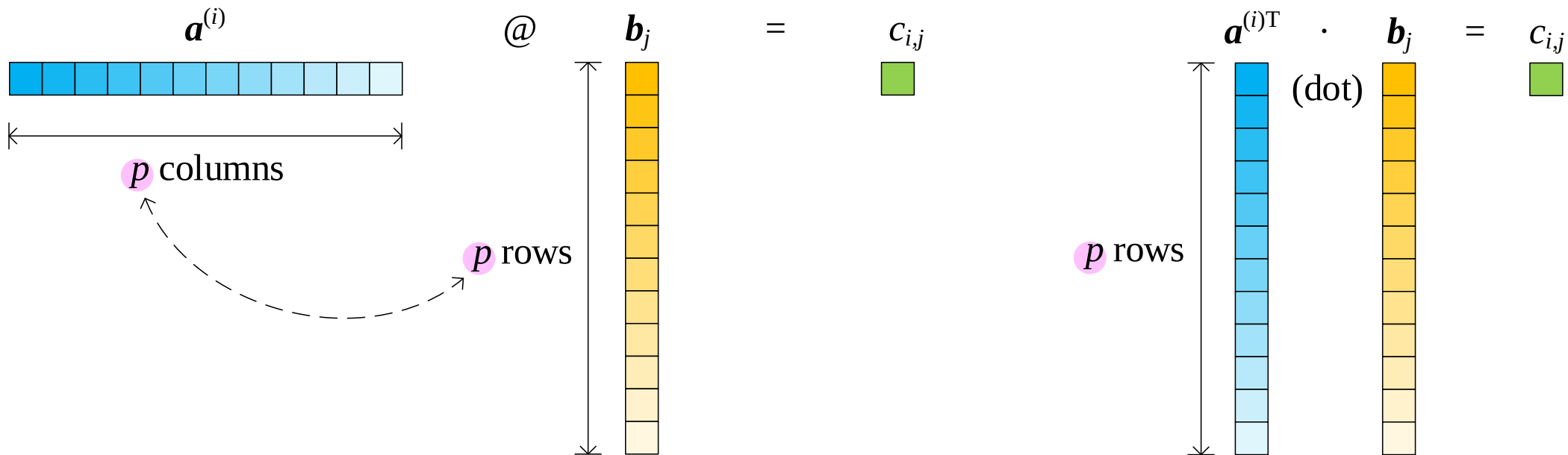
Matrix Multiplication Can Produce a Scalar

A $1 \times p$ row vector times a $p \times 1$ column vector gives a 1×1 matrix — a scalar

- This is worth remembering: matrix multiplication doesn't always yield a “big matrix” — it can be a single number
- The product of a row vector and a column vector can also be written as a vector dot product

Figure 5 · Matrix Product $\mathbf{a}^{(i)} @ \mathbf{b}_j = c_{i,j}$

- A $1 \times p$ row vector $\mathbf{a}^{(i)}$ times a $p \times 1$ column vector $\mathbf{b}_j$ yields a scalar
- The full product $\mathbf{C} = \mathbf{AB}$ amounts to $m \times n$ such multiplications — $m \times n$ vector dot products



A Worked Example

Let $A = [1,2,3; 4,5,6]$ and $B = [1,4; 2,5; 3,6]$

- A has shape 2×3 and B has shape 3×2 , so the product $C = AB$ has shape 2×2
- $(2 \times 3) @ (3 \times 2)$ — the middle 3 cancels, leaving a product of shape 2×2

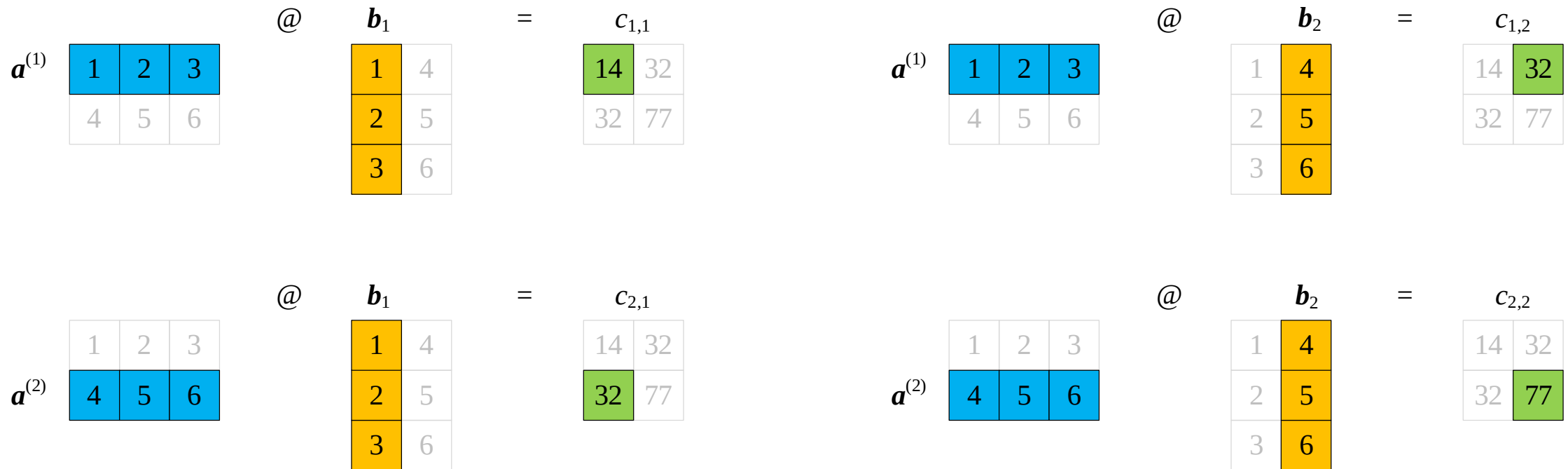
Figure 6 · Product AB; A has shape 2×3, B has shape 3×2

- Element by element:
- $c_{1,1}=1\times1+2\times2+3\times3=14$,
- $c_{1,2}=1\times4+2\times5+3\times6=32$
- $c_{2,1}=4\times1+5\times2+6\times3=32$,
- $c_{2,2}=4\times4+5\times5+6\times6=77$, giving $C = [14,32; 32,77]$

$$\begin{array}{c} \mathbf{A} \\ \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline \end{array} \end{array} \quad @ \quad \begin{array}{c} \mathbf{B} \\ \begin{array}{|c|c|} \hline 1 & 4 \\ \hline 2 & 5 \\ \hline 3 & 6 \\ \hline \end{array} \end{array} = \begin{array}{c} \mathbf{C} \\ \begin{array}{|c|c|} \hline 14 & 32 \\ \hline 32 & 77 \\ \hline \end{array} \end{array}$$

Figure 7 · Computing Each Element of the Product AB

- $\mathbf{a}^{(1)} @ \mathbf{b}_1 = 14$ and $\mathbf{a}^{(1)} @ \mathbf{b}_2 = 32$ form $\mathbf{C}$'s first row
- $\mathbf{a}^{(2)} @ \mathbf{b}_1 = 32$ and $\mathbf{a}^{(2)} @ \mathbf{b}_2 = 77$ form $\mathbf{C}$'s second row



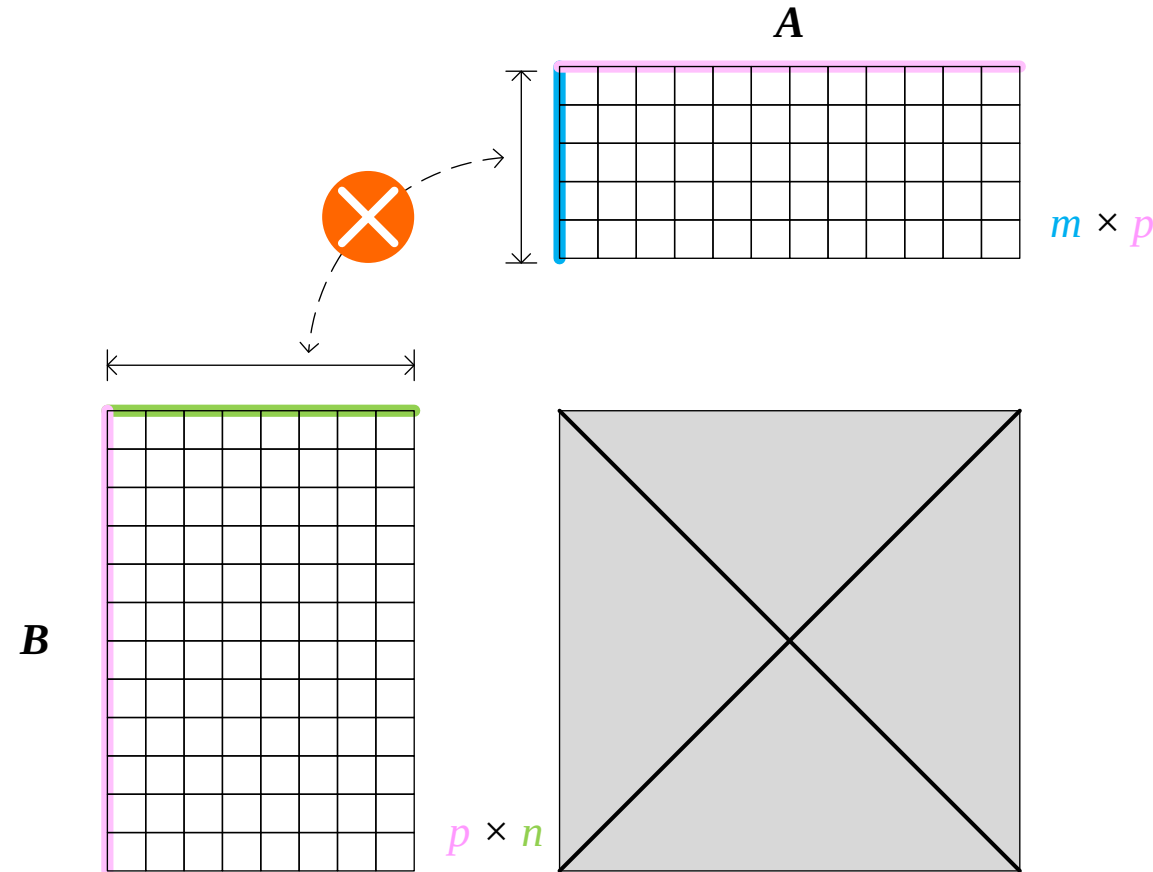
Matrix Multiplication Is Non-Commutative

Order matters in matrix multiplication and cannot be swapped freely: $AB \neq BA$ (non-commutative)

- $A @ B$ being valid does not guarantee that $B @ A$ is valid
- Even when $B @ A$ is valid, the results of $A @ B$ and $B @ A$ need not match

Figure 8 · $B @ A$ Violates the Shape-Matching Rule

- A has shape $m \times p$ and B has shape $p \times n$; $B @ A$ requires B 's n columns to equal A 's m rows
- If $n \neq m$, $B @ A$ cannot be performed at all — shape matching is the first hurdle



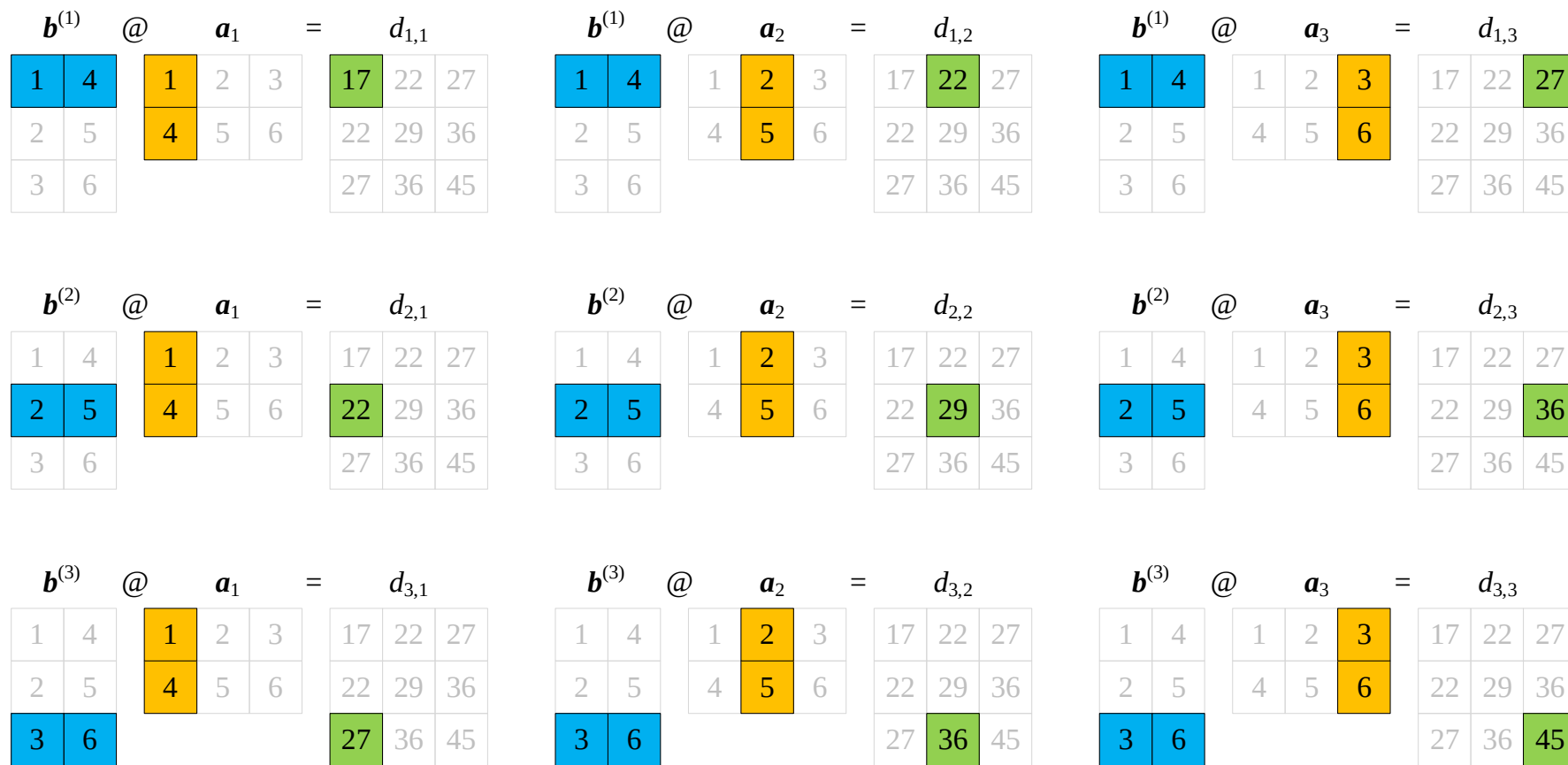
B @ A Clearly Differs from A @ B

$$\mathbf{D} = \mathbf{B} @ \mathbf{A} = [1,4; 2,5; 3,6] @ [1,2,3; 4,5,6] = [17,22,27; 22,29,36; 27,36,45]$$

- $(3 \times 2) @ (2 \times 3)$ — the middle 2 cancels, so $\mathbf{D}$ has shape 3×3
- $\mathbf{D}$'s shape differs entirely from $\mathbf{C} = \mathbf{AB}$'s shape (2×2) , a direct illustration that $\mathbf{AB} \neq \mathbf{BA}$

Figure 9 · Computing Each Element of $B @ A$

- Each of D's 9 entries is computed as the dot product of one of B's rows with one of A's columns
- A closer look shows that both AB and BA happen to be symmetric matrices in this example



A Custom Python Function for Matrix Multiplication

Three nested for loops reimplement matrix multiplication, revealing what the @ operator does under the hood

- The outer loop sweeps $\mathbf{A}$'s rows, the middle loop sweeps $\mathbf{B}$'s columns, the inner loop sums elementwise products
- If $\mathbf{A}$'s columns don't match $\mathbf{B}$'s rows, the function raises an error immediately

A Custom Function: Matrix Multiplication via Triple Nested Loops

matrix_multiplication(A, B)

reimplements matrix multiplication from scratch

- `m, p_A = A.shape` and `p_B, n = B.shape` read off the shapes of both matrices
- if `p_A != p_B` checks for a shape mismatch and raises a `ValueError` if the shapes don't match
- `C = np.zeros((m, n))` initializes the result matrix with shape `(m, n)`, filled with zeros
- Three nested loops over `i, j, k` sweep rows, columns, and the summed dimension: `C[i,j] += A[i,k]*B[k,j]`

a

b

c

d

e

f

g

h

i

```

## 初始化
import numpy as np

## 自定义函数
def matrix_multiplication(A, B):

    # 获取矩阵 A 和 B 的形状
    m, p_A = A.shape
    p_B, n = B.shape

    # 检测矩阵形状是否符合矩阵乘法规则
    if p_A != p_B:
        raise ValueError('Dimensions do not match')

    # 初始化结果矩阵 C, 形状 (m, n), 初始值设为 0
    C = np.zeros((m, n))

    # 进行矩阵乘法计算, 使用三层 for 循环
    for i in range(m):          # 遍历 A 的行
        for j in range(n):      # 遍历 B 的列
            for k in range(p_A): # 遍历 A 的列 / B 的行
                C[i, j] += A[i, k] * B[k, j] # 逐元素累加

    return C

## 定义矩阵
A = np.array([[1, 2, 3],
              [4, 5, 6]])
B = A.T

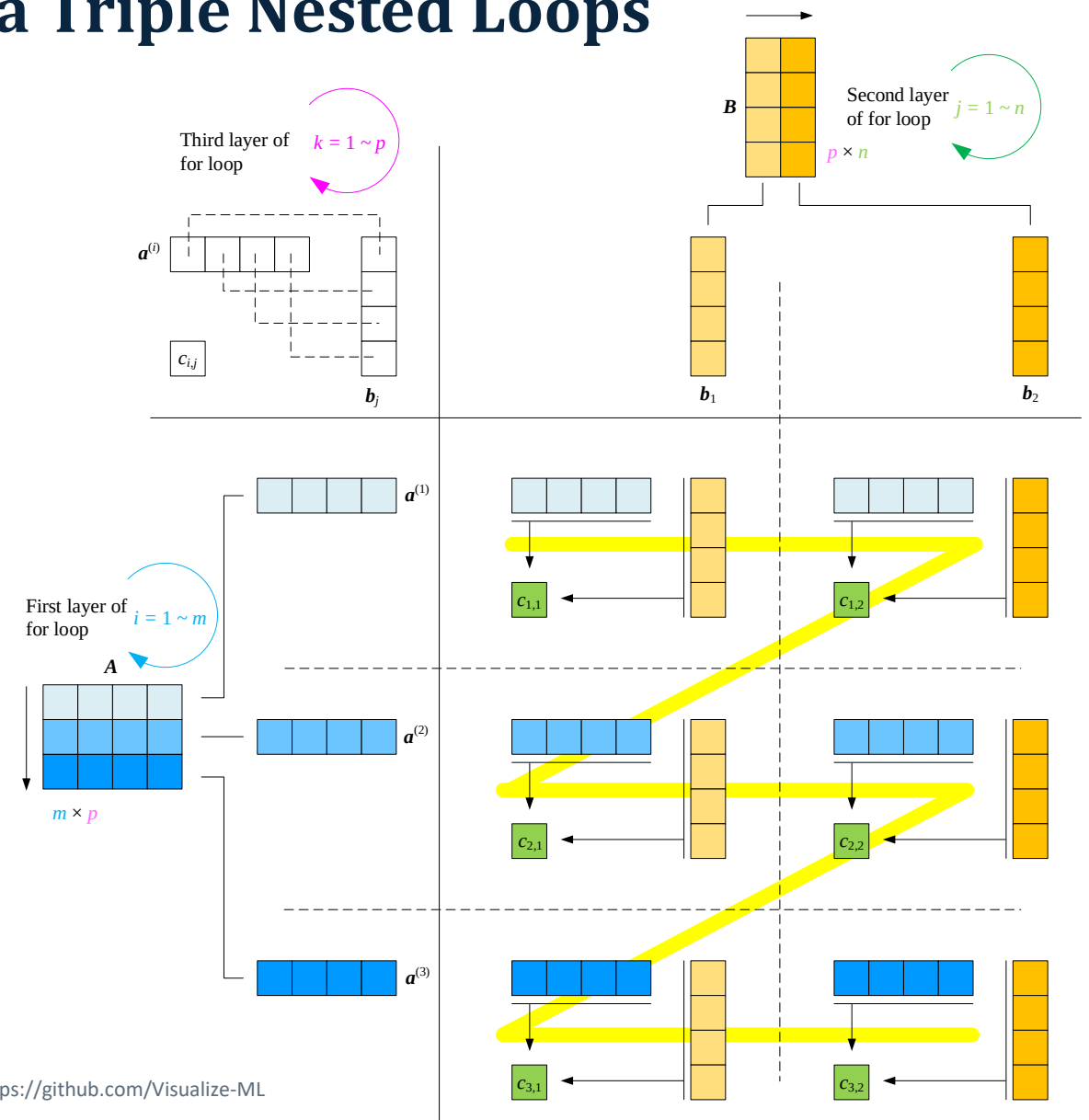
## 矩阵乘法
matrix_multiplication(A, B)
matrix_multiplication(B, A)

```



Figure 10 · The Rule Behind AB, via Triple Nested Loops

- The first loop i sweeps A 's rows; the second loop j sweeps B 's columns
- The third loop k multiplies and accumulates elementwise — exactly a row-column dot product



The Importance of Matrix Multiplication Cannot Be Overstated

This section only scratches the surface — shape matching and elementwise dot products are the starting point for everything

- Matrix multiplication is one of the most fundamental operations in data science and machine learning
- Mastering how the @ operator maps onto triple nested loops builds intuition for more advanced matrix operations

Key Terms in This Section

Matrix Multiplication *Two matrices combine into a new matrix, written AB or $A @ B$*

Dot Product *Elementwise products of a row and a column vector, summed*

Associative Law *Matrix multiplication satisfies $(AB)C = A(BC)$*

Scalar Result *The product of a 1×1 matrix can be treated as an ordinary number*

Heatmap Visualization *Encodes matrix values as color intensity for quick visual comparison*

Shape Compatibility *The left matrix's columns must equal the right matrix's rows*

Commutative Law *Generally does not hold for matrix multiplication: $AB \neq BA$*

Distributive Law *Matrix multiplication satisfies $A(B+C) = AB + AC$*

The @ Operator *NumPy's concise syntax for matrix multiplication, equivalent to `np.matmul()`*

Triple Nested Loop *The underlying algorithm behind a hand-written matrix multiplication*

FURTHER LEARNING

Companion Code and Further Reading

Further Reading · LA_02_04_02.ipynb

Full code for computing matrix products element by element

Further Reading · LA_02_04_03.ipynb

A custom Python function implementing matrix multiplication via triple loops

Recommended · The Power of Matrices

A deeper look at matrix multiplication in data science and machine learning

Open-Source Repository

All code and figures for this book: <https://github.com/Visualize-ML>

Questions Q1 – Q9

Q1 Decide whether each pair of matrices below can support $A @ B$ or $B @ A$: $A_{2 \times 2} = [1, 2; 3, 4]$ and $B_{2 \times 2} = [5, 6; 7, 8]$; $A_{1 \times 3} = [1, 2, 3]$ and $B_{3 \times 1} = [4; 5; 6]$; $A_{3 \times 2} = [1, 2; 3, 4; 5, 6]$ and $B_{2 \times 2} = [7, 8; 9, 10]$; $A_{2 \times 3} = [1, 2, 3; 4, 5, 6]$ and $B_{3 \times 2} = [7, 8; 9, 10; 11, 12]$; $A_{2 \times 2} = [1, 2; 3, 4]$ and $B_{2 \times 1} = [5; 6]$.

Q3 Given column vector $a = [1; 2]$, compute $a^T @ a$ and $a @ a^T$.

Q5 Given $A = [1, 2; 3, 4]$ and $b = [1; 2]$, compute $A @ b$ and $b^T @ A^T$.

Q7 Use NumPy to compute the matrix products shown in each panel of Figure 9, and verify each with a vector dot product.

Q9 Randomly generate $A(4 \times 6)$ and $B(6 \times 3)$ with NumPy, and verify that $A @ B$'s shape follows the “middle dimension cancels” rule.

Q2 Each of the following 10 matrix pairs supports both $A @ B$ and $B @ A$. Use hand calculation or code to decide whether $A @ B$ equals $B @ A$, covering identity, symmetric, all-ones, diagonal, and rotation matrices.

Q4 Given column vectors $a = [1; 2]$ and $b = [1; 2; 3]$, compute $a @ b^T$ and $b @ a^T$.

Q6 Given $A = [1, 2; 3, 4; 5, 6]$, compute $A @ A^T$ and $A^T @ A$.

Q8 Modify Code 2 by swapping the first and second for-loop layers, and check whether the result changes.

Questions Q10 – Q19

Q10 Write a function `check_shape(A, B)` that decides whether $A @ B$ is valid, returning the result's shape if so and a message otherwise.

Q12 Use NumPy to verify that matrix multiplication distributes over addition: $A @ (B + C)$ versus $A @ B + A @ C$.

Q14 Write a function `dot_product_check(A, B)` that verifies every element of $A @ B$ via the corresponding row-column dot product.

Q16 Use `seaborn.heatmap()` to plot an asymmetric matrix A alongside its transpose A^T , and compare the two.

Q18 Use NumPy to verify that multiplying a diagonal matrix D by any same-size square matrix A , $D @ A$, is equivalent to scaling each row of A .

Q11 Use NumPy to verify that matrix multiplication is associative: $(A @ B) @ C$ versus $A @ (B @ C)$.

Q13 Generate a random square matrix A and verify that $A @ I$ and $I @ A$ both equal A (I being the identity of matching size).

Q15 Time NumPy's `@` operator against your own triple-loop function on a large matrix (e.g. 200×200), and compare their speed.

Q17 Given a square matrix A , check whether $A @ A$ (A^2) and $A @ A @ A$ (A^3) always keep the same shape.

Q19 Generate two 2×2 rotation matrices, and verify that multiplying them is equivalent to a single rotation by the combined angle.

Questions Q20 – Q28

Q20 Write a function `is_result_symmetric(A)` that verifies $A @ A^T$ and $A^T @ A$ are always symmetric, testing with repeated random matrices.

Q22 Generate a random 5×5 matrix A , compute the trace of $A @ A$, and compare it with the sum of A 's eigenvalues (`numpy.linalg.eigvals()`).

Q24 Use `seaborn.heatmap()` to visualize this section's two examples ($A @ B$ and $B @ A$) side by side, comparing shape and values.

Q26 Use NumPy to verify that multiplying a zero matrix O by any matrix A , $A @ O$ and $O @ A$, always yields a correctly-shaped zero matrix.

Q28 Build a simple “matrix multiplication visualizer” that, given two small matrices, prints out each step's row-vector-dot-column-vector computation for c_{ij} .

Q21 Use NumPy to check whether `np.matmul()`, `np.dot()`, and the `@` operator give identical results for 2-D matrix multiplication.

Q23 Write a function `batch_matmul(A_list, B_list)` that takes two lists of matrices and returns a list of their corresponding products.

Q25 Given a rectangular matrix A ($m \times p$, $m \neq p$), compare the shapes of $A @ A^T$ and $A^T @ A$, and describe a use case for each (e.g. covariance matrices).

Q27 Write a function `matmul_report(A, B)` that automatically checks whether $A @ B$ and $B @ A$ are valid, reports their shapes, and notes whether they're equal.

10 Prompts to Explore Further (1/2)

- 1 Explain why the shape-matching rule 'the middle dimension cancels' works, and connect it to the definition of a dot product.
- 2 Why can matrix multiplication produce a 1×1 matrix (a scalar)? How does this relate to the vector inner product?
- 3 Prove that matrix multiplication is generally non-commutative, and give a special case where $AB = BA$ holds (e.g. the identity matrix).
- 4 Explain the time complexity of the triple-loop matrix multiplication, and why NumPy's `@` operator is so much faster than a hand-written loop.
- 5 Design a Python experiment that verifies matrix multiplication is associative, $(AB)C = A(BC)$.

10 Prompts to Explore Further (2/2)

6

Explain why matrix multiplication is so often used for batch computation in machine learning, such as the forward pass of a fully connected layer.

7

Explain, from a geometric viewpoint, why multiplying rotation matrices is equivalent to adding their angles.

8

Explain why AA^T and A^TA are always symmetric matrices, and describe their role in statistics (e.g. covariance matrices).

9

If you swap the order of the i, j, k loops in the custom function, does the result change? Does the running time change? Explain why.

10

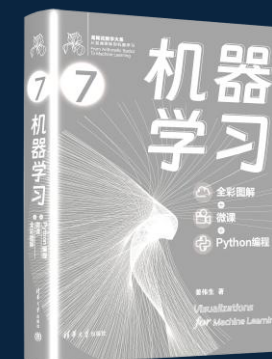
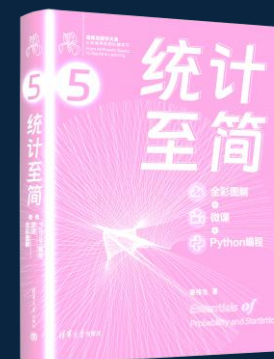
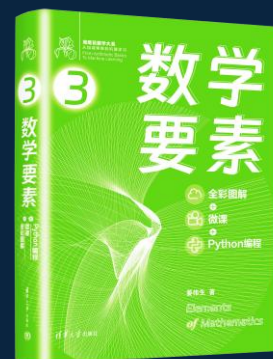
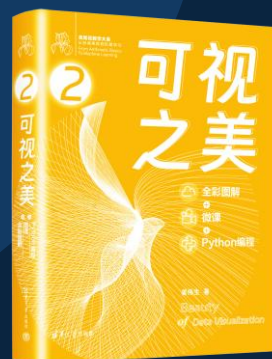
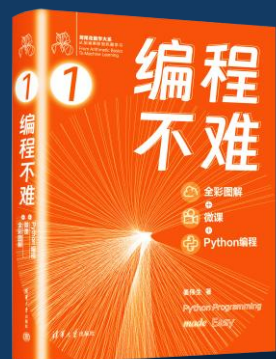
Explain the value of visualizing matrix multiplication as a heatmap, and how color changes make the result intuitive to read.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

